

EECE 5496 Week 3:

September 16, 2026

Pre-req and Note

Make sure you have completed the Week 2 In-Class Model. We will make changes to the Week 2 model; thus, you may want to make a copy of your project if needed.

1 Instructions for Developing In-Class Model

1. Add the following requirements to the model:
 - Soldiers with Code 0 shall be 10 % of the battalion
 - Soldiers with Code 1 shall be 30% of the battalion
 - Soldiers with Code 2 shall be 25% of the battalion
 - Soldiers with Code 3 shall be 35% of the battalion
2. On the Battalion block, create new value properties for Number of Soldiers and the percentages of each code. For example, you may define them as Nsoldier, code0_perc, code1_perc, code2_perc, code3_perc. The last four value properties are used to store the per-

centage of soldiers in the battalion with sensor code i where $i = 0, 1, 2, 3$. Set appropriate type for each value property (e.g., Integer, Real, etc).

3. Set the default value for `Nsoldier`, and also make the multiplicity of the soldier to the same value as `Nsoldier`.
4. Add the created requirements to the BDD which shows your Battalion block. Add “Satisfy” relationships between the value properties you created and the requirements. This is to show which model element fulfills the requirement; thus, the arrowhead will point at the requirement.
5. For each soldier, we’ll assign a sensor code (0-3) using a random number. To do this, add a new value property for a random number (e.g., `rand_a : Real`) to the soldier block. Apply Uniform stereotype to it and set `min = 0.0` and `max = 1.0`. With this, the tool will draw a uniformly distributed number for the value property when you initiate the simulation.
6. To assign Sensor Code given this randomly generated number, we will demonstrate the method with the constraint block. Create a Constraint Block with two parameters: one for input and other for output. Add the following code to the Constraint specification with language **Groovy**:

```
if(a < 0.1) {  
    code = 0;  
}
```

```

else if (a < 0.4) {
    code = 1;
}
else if (a < 0.65) {
    code = 2;
}
else {
    code = 3;
}

```

Note that if-else conditions are from the requirements.

7. Make sure that the soldier block is composed of the constraint block that you just created. That means every soldier will own the code generator mechanism.

Fig. 1 shows an example of the BDD after completing Steps 1-7.

8. Create the parametric diagram from the soldier block. In the diagram, add two value properties from the soldier (rand_a and SensorCode), the constraint, and the constraint parameters. Connect the constraint parameters and soldier value properties accordingly.
9. Next, we'll update the Battalion activity diagram. Note that for your expansion region action, you will need at least the read structural feature for "sensor-Code" and the opaque action for computing NSensorA and NSensorB from week 2 model.

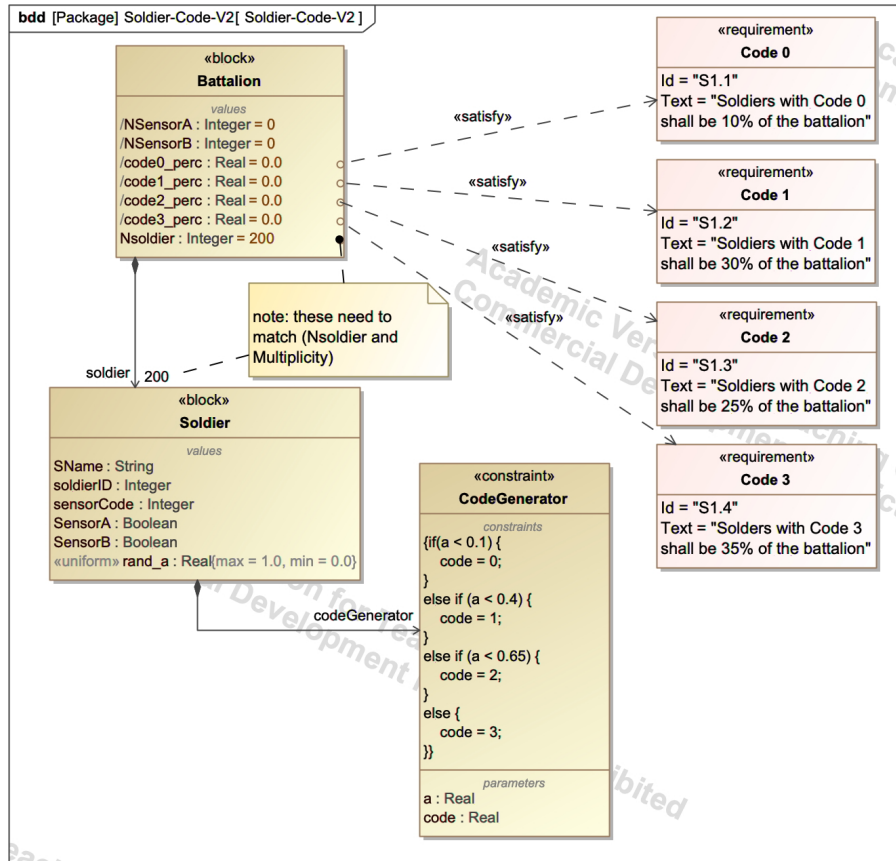


Figure 1: Examples of the BDD after steps 1-7

10. Add a decision node to the expansion region. Also, define four output expansion nodes - each of these node will be used to keep track of code 0-3 occurrences. The decision node will need input which is "sensorCode" and will have four output flows, each connects to the output expansion node. Set guard condition for the decision node's output flows to "0", "1", "2", and "3" accordingly. We want this guard to be treated 'true' when the token value (incoming flow to the decision node) matches the guard value - to

do this, go to ‘Options’ → ‘Project’ → ‘General’ → ‘Simulation’. Under ‘fUML Engine’, set ‘Use fUML Decision semantics’ to true.

11. Next, add four opaque actions outside of the expansion region. Each of these opaque actions will take input from the output expansion node. Make sure to set the multiplicity to ‘1..*’. For the body of the opaque action, add $\text{count}(\text{in-pin name}) * 100 / \text{Nsoldier}$ with language **Built-in Math**.

Note that you need to update the above opaque action body with your battalion’s value property as well as your input pin name.

12. End your activity once all of the opaque actions computes.

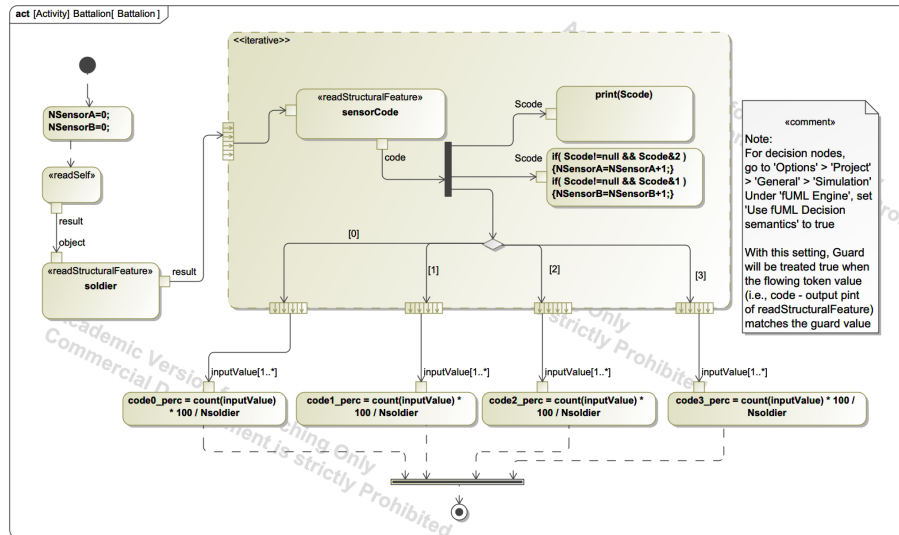


Figure 2: Updated Activity diagram after Steps 9-12

13. Run simulation with your Battalion block. In the Simulation window, your Variables tab will list the values for the value properties. The four value properties that have “satisfy” relationships will have color codes: Red means the requirement fails and Green means the requirement fulfills.

If you don't see color codes, go to 'Options' → 'Project' → 'General' → 'Requirements'. Set all three properties under Requirements to true.

14. To save the simulation results, you can create an instance table with the Battalion block as Classier and set Scope if needed. To display 'Verification Status' that shows Pass or Fail status for the requirements, Click 'Legend' Icon from the diagram toolbar, and select 'Apply Legends' and check 'Verification Status.'

Keep in mind that when you change the value of the NSoldier value property within the instance table, you also need to change the multiplicity of the soldier part property to make sure they match.